

Лабораторная работа №17. Основы алгоритмов и блок-схем

Цель лабораторной работы:

- Понять, что такое алгоритм и его свойства
- Научиться составлять блок-схемы алгоритмов
- Освоить работу с расширением Draw.io в VS Code
- Научиться переводить блок-схемы в код на языке C#
- Решать практические задачи с использованием алгоритмов

Шаг 1. Подготовка проекта

1. Откройте **Visual Studio Code**
2. Откройте встроенный терминал
3. Перейдите в каталог с документами
4. Создайте директорию для лабораторной работы:

Lab17_Algorithms_FIO (где FIO — ваша фамилия)

5. Перейдите в созданную директорию
6. Создайте:

файлы:

- README.md
- .gitignore

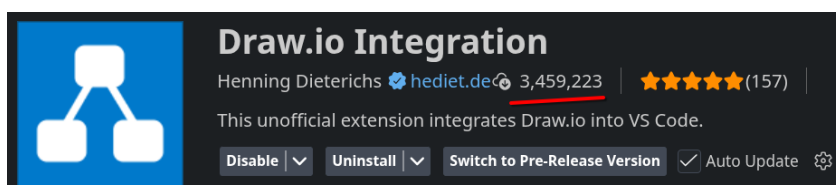
папки:

- *diagrams*
- *examples*
- *screenshots*
- *code*

7. Откройте созданный проект в **Visual Studio Code**

Установка расширения draw.io

1. В **Visual Studio Code** откройте панель расширений (**Ctrl + Shift + X**)
2. Найдите расширение: **Draw.io Integration**



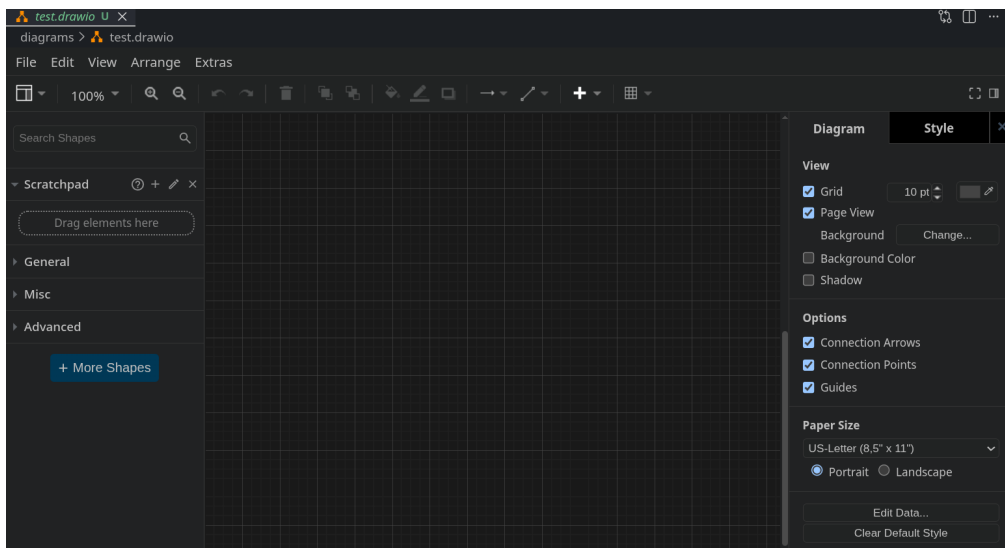
3. Нажмите кнопку **Install**
4. После установки перезагрузите VS Code

Проверка установки:

1. Создайте тестовый файл с расширением **.drawio**:

touch diagrams/test.drawio

2. Откройте файл — должен запуститься графический редактор **Draw.io**



3. Если редактор открылся — установка прошла успешно!

Инициализация Git-репозитория

1. Инициализируйте Git-репозиторий:

git init

2. Внутри файла **.gitignore** него пропишите:

***.tmp**

.DS_Store

.vscode/

***.bak**

bin/

obj/

.vs/

3. Добавьте файлы в индекс:

git add .

4. Создайте первый коммит:

git commit -m "Initial commit: project structure and Draw.io setup"

5. Создайте и переключитесь на ветку **main**:

git branch -M main

Создание удалённого репозитория

1. Создайте пустой **public-репозиторий** на GitHub с именем:

Lab17_Algorithms

2. Привяжите локальный репозиторий к удалённому:

git remote add origin <URL_репозитория>

3. Отправьте файлы:

git push -u origin main

4. Сделайте скриншот терминала с выполненными командами и сохраните изображение в папку **screenshots** под именем:

gitPushLab17_Algorithms_FIO.png

5. Добавьте изменения и отправьте их:

git add .

git commit -m "Added push screenshot"

git push

Шаг 2. Что такое алгоритм?

Алгоритм — это точная конечная последовательность действий, направленная на достижение определённой цели или решение конкретной задачи.

Свойства алгоритма:

1. **Дискретность** — алгоритм состоит из отдельных шагов
2. **Детерминированность** — каждый шаг однозначно определён
3. **Конечность** — алгоритм завершается за конечное число шагов
4. **Массовость** — применим для целого класса задач
5. **Результативность** — всегда приводит к результату

Примеры алгоритмов из жизни

Пример 1: Приготовление чая

Задача: Приготовить чашку чая

Алгоритм:

1. Взять чайник
2. Налить в чайник воду
3. Поставить чайник на плиту
4. Включить плиту
5. Дождаться закипания воды

6. Взять чашку
7. Положить в чашку чайный пакетик
8. Налить кипятка в чашку
9. Добавить сахар (по желанию)
10. Размешать
11. Чай готов!

Пример 2: Переход дороги

Задача: Безопасно перейти дорогу

Алгоритм:

1. Подойти к пешеходному переходу
2. Посмотреть налево
3. Посмотреть направо
4. Если машин нет — перейти
5. Если машины есть — подождать
6. Повторить шаги 2-5 до перехода

Создание первой блок-схемы

1. Создайте файл для первой диаграммы:

touch diagrams/step2_tea_algorithm.drawio

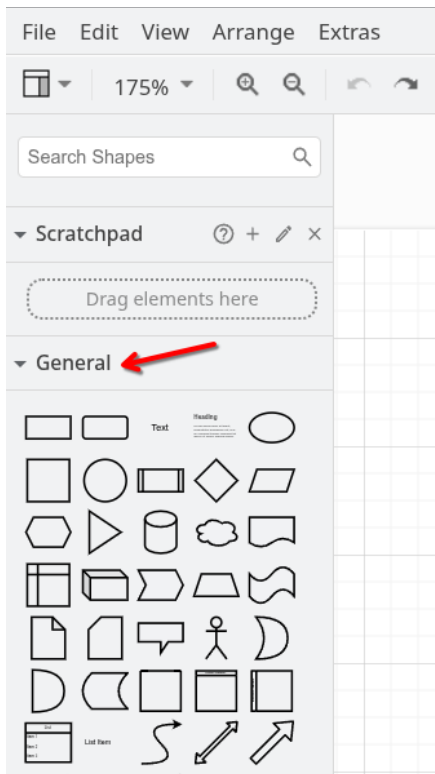
2. Откройте файл и нарисуйте блок-схему приготовления чая, используя следующие элементы:

Основные элементы блок-схем:

Элемент	Форма	Назначение
Начало/Конец	Овал	Начало или завершение алгоритма
Процесс	Прямоугольник	Выполнение действия
Решение	Ромб	Условие (да/нет)
Ввод/Вывод	Параллелограмм	Ввод данных или вывод результата
Стрелка	Линия со стрелкой	Направление выполнения

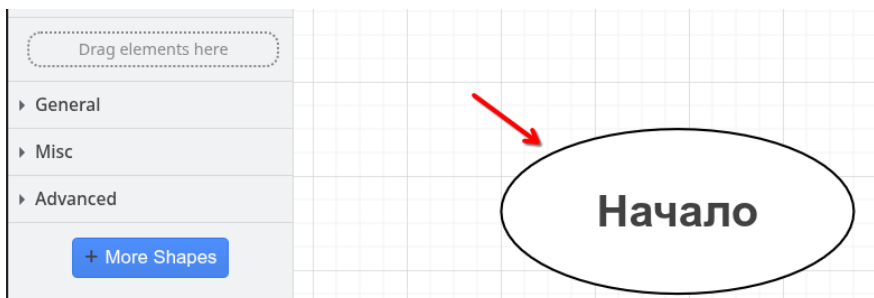
Инструкция по рисованию:

1. Откройте файл **step2_tea_algorithm.drawio**
2. На панели слева в разделе **General**(Блок-схемы)



P.S. если хотите светлую тему, то в settings.json нужно добавить строку:
"hediet.vscode-drawio.appearance": "light",

3. Перетащите **Ellipse** на рабочую область — подпишите **"Начало"**



4. Добавьте прямоугольники для каждого действия

5. Соедините элементы стрелками



6. В конце добавьте овал "Конец"

7. Сохраните файл (Ctrl + S)

Пример структуры:



Практическое задание

1. Создайте блок-схему алгоритма приготовления чая
2. Сделайте скриншот блок-схемы и сохраните в папку **screenshots** как:

step2_teaAlgorithmLab17_FIO.png

3. Выполните команды:

git add .

git commit -m "Step 2: introduction to algorithms and first flowchart"

git push

Шаг 3. Алгоритм с условием (ветвление)

Теперь создадим алгоритм с условием — когда программа принимает решение.

Жизненный пример: Выбор одежды по погоде

Задача: Выбрать одежду в зависимости от температуры

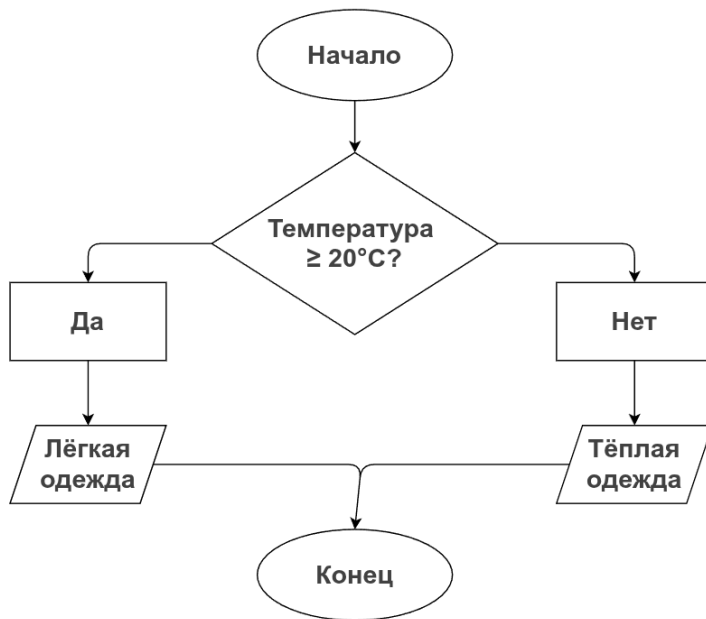
Алгоритм:

1. Начало
2. Посмотреть на термометр
3. Узнать температуру
4. **ЕСЛИ** температура $\geq 20^{\circ}\text{C}$
 - ◆ ТО надеть лёгкую одежду
5. **ИНАЧЕ**
 - ◆ Надеть тёплую одежду
6. Конец

Создайте новый файл:

touch diagrams/step3_weather_algorithm.drawio

Откройте файл и нарисуйте блок-схему:



Важно: В ромбе всегда задаётся вопрос, на который можно ответить ДА или НЕТ.

Перевод в код на языке C#

Теперь переведём этот алгоритм в программный код.

Перейдём в директорию **code** и создадим в ней файл **Step3_WeatherChoice.cs**

cd code

dotnet new console --name Step3_WeatherChoice

cd Step3_WeatherChoice/

В файл **Program.cs** добавьте следующий код:

```
1 //Начало
2 Console.WriteLine("Выбор одежды по погоде");
3 Console.Write("Введите текущую температуру (°C): ");
4 int temperature = Convert.ToInt32(Console.ReadLine());
5 // Условие (ветвление)
6 if (temperature >= 20)
7 {
8     // Условие ИСТИННО (ДА)
9     Console.WriteLine("Наденьте лёгкую одежду");
10 }
11 else
12 {
13     // Условие ЛОЖНО (НЕТ)
14     Console.WriteLine("Наденьте тёплую одежду");
15 }
16 // Конец
17 Console.WriteLine("Хорошего дня!");
```

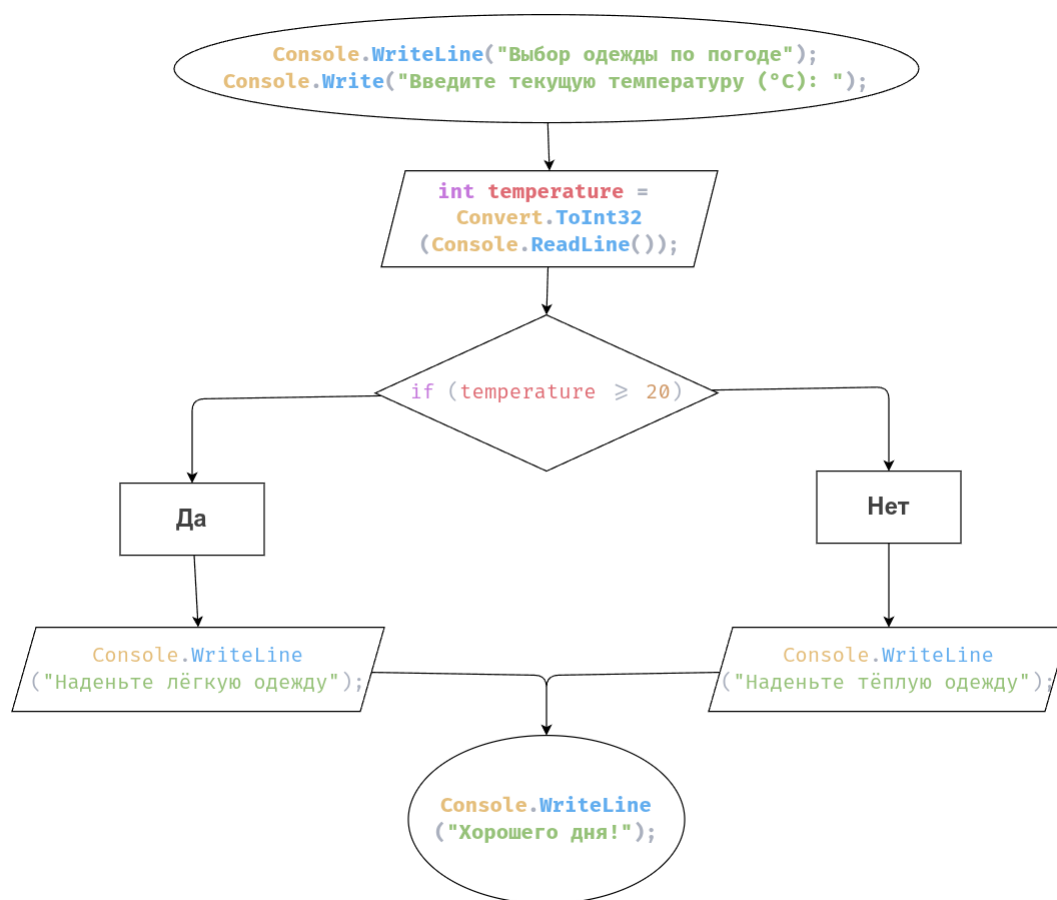
Запустите в терминале код:

dotnet run

В зависимости от ввода, вы получите соответствующий вывод:

```
[denis@denis-Desktop:~/P/I/L/c/Step3_We
>$ dotnet run
Выбор одежды по погоде
Введите текущую температуру (°C): 23
Наденьте лёгкую одежду
Хорошего дня!
```

Нарисуем в **step3_weather_algorithm** ещё одну блок-схему:



Выполните команды:

cd ../../

git add .

git commit -m "Step 3: conditional algorithm and if/else in C#"

git push

Самостоятельные задания

Практическое задание №1

1. Нарисуйте блок-схему выбора одежды

2. Создайте программу на C#

3. Сделайте два скриншота:

- **task_weatherDiagramLab17_FIO.png** (блок-схема)
- **task_weatherCodeLab17_FIO.png** (код программы)

Выполните команды:

git add .

git commit -m "Step 3: conditional algorithm and if/else in C#"

git push

Шаг 4. Переменные и типы данных

Прежде чем двигаться дальше, разберёмся с **переменными** — это ячейки памяти для хранения данных.

Что такое переменная?

Переменная — это именованное место в памяти компьютера, где хранится значение, которое может изменяться.

Аналогия: Переменная — это коробка с наклейкой (именем), в которую можно положить значение.

Основные типы данных в C#

Тип	Описание	Пример значения
int	Целое число	42, -10, 0
double	Дробное число	3.14, -0.5, 2.0
string	Текст (строка)	"Привет", "C#"
bool	Логическое значение	true, false
char	Один символ	'A', '5', '@'

Жизненный пример: Калькулятор чаевых

Задача: Рассчитать сумму чаевых в ресторане

Алгоритм:

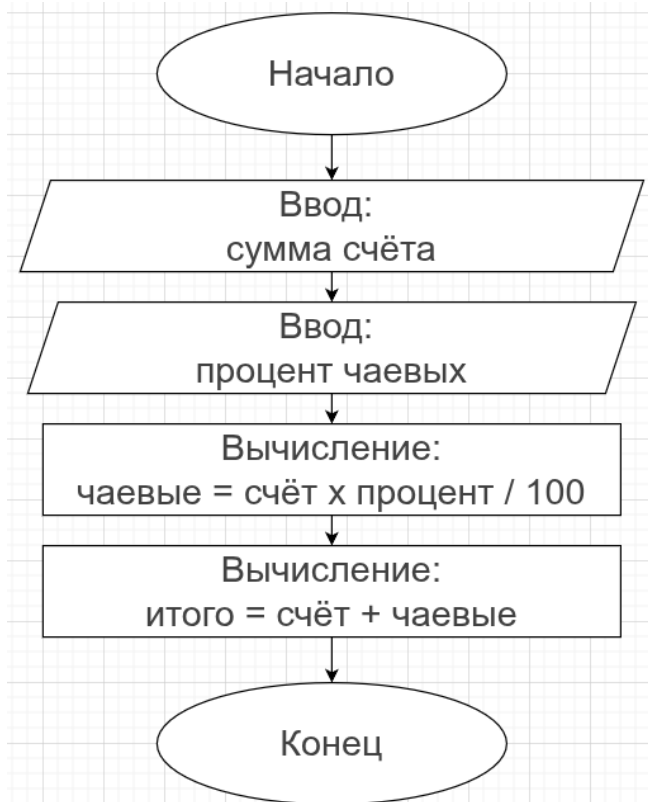
1. Начало
2. Узнать сумму счёта
3. Узнать процент чаевых
4. Рассчитать сумму чаевых: $\text{чаевые} = \text{счёт} \times (\text{процент} / 100)$
5. Рассчитать общую сумму: $\text{итого} = \text{счёт} + \text{чаевые}$
6. Вывести результат

7. Конец

Создайте файл для блок-схемы:

touch diagrams/step4_tips_calculator.drawio

Нарисуем блок-схему с вычислениями:



Создайте файл с кодом:

Перейдём в директорию **code** и создадим в ней файл **Step4_TipsCalculator.cs**

cd code/

dotnet new console --name Step4_TipsCalculator

cd Step4_TipsCalculator/

В файл **Step4_TipsCalculator/Program.cs** добавьте следующий код:

```
// Начало
Console.WriteLine("Калькулятор чаевых");
// Объявление переменных и ввод данных
Console.Write("Введите сумму счёта (руб): ");
double bill = Convert.ToDouble(Console.ReadLine());
Console.Write("Введите процент чаевых (например, 10): ");
double tipPercent = Convert.ToDouble(Console.ReadLine());
// Вычисления
double tipAmount = bill * (tipPercent / 100);
double totalAmount = bill + tipAmount;
```

```
// Вывод результата
Console.WriteLine("\nРезультат:");
Console.WriteLine($"Сумма счёта: {bill} руб");
Console.WriteLine($"Чаевые ({tipPercent}%): {tipAmount} руб");
Console.WriteLine($"Итого к оплате: {totalAmount} руб");
// Конец
Console.WriteLine("Спасибо за использование калькулятора!");
```

Запустите в терминале код:

dotnet run

В зависимости от ввода, вы получите соответствующий вывод:

```
>$ dotnet run
Калькулятор чаевых
Введите сумму счёта (руб): 550
Введите процент чаевых (например, 10): 7

Результат:
Сумма счёта: 550 руб
Чаевые (7%): 38,500000000000001 руб
Итого к оплате: 588,5 руб
Спасибо за использование калькулятора!
```

Практическое задание

1. Нарисуйте блок-схему калькулятора чаевых
2. Запустите программу и протестируйте с разными значениями
3. Сделайте три скриншота:
 - **step4_tipsDiagramLab17_FIO.png** (блок-схема)
 - **step4_tipsCodeLab17_FIO.png** (код)
 - **step4_tipsRunLab17_FIO.png** (результат выполнения)
4. Выполните команды:

cd ../../

git add .

git commit -m "Step 4: variables, data types and calculations"

git push

Шаг 5. Множественное ветвление (if-else if-else)

Часто нужно проверить несколько условий подряд. Для этого используется конструкция **if-else if-else**.

Жизненный пример: Оценка успеваемости

Задача: Определить оценку студента по баллам

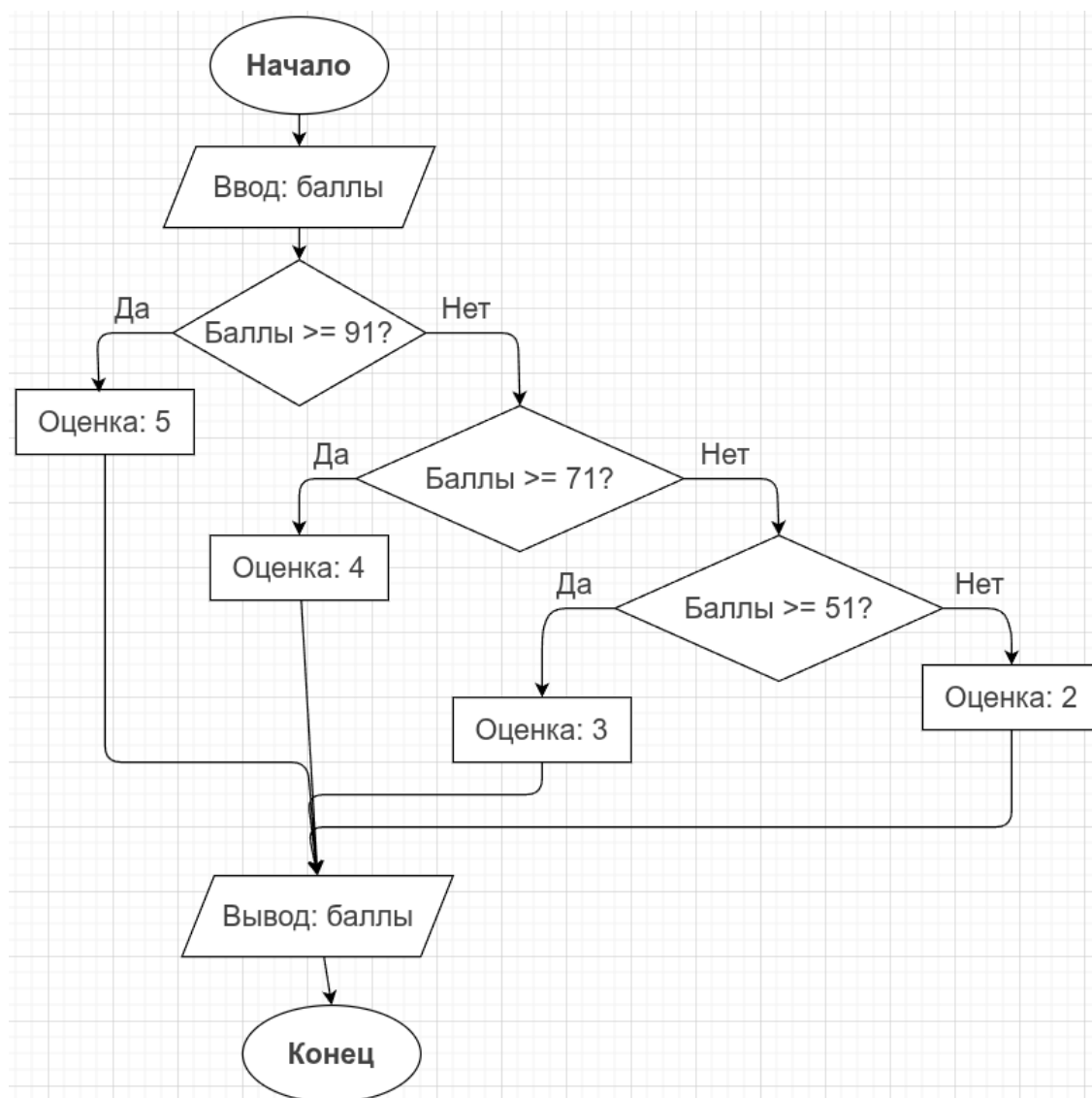
Правила:

- 91-100 баллов → "Отлично" (5)
- 71-90 баллов → "Хорошо" (4)
- 51-70 балла → "Удовлетворительно" (3)
- 0-50 баллов → "Неудовлетворительно" (2)

Создайте файл:

touch diagrams/step5_grade_check.drawio

Нарисуем блок-схему с множественными условиями:



Создайте файл с кодом:

cd code/

dotnet new console --name Step5_GradeCheck

cd Step5_GradeCheck/

В файл **Step5_GradeCheck/Program.cs** добавьте следующий код:

```
// Начало
Console.WriteLine("Определение оценки по баллам");
// Ввод данных
Console.Write("Введите количество баллов (0-100): ");
int score = Convert.ToInt32(Console.ReadLine());
// Переменная для хранения оценки
string grade;
int numericGrade;
// Множественное ветвление
if (score >= 91 && score <= 100)
{
    grade = "Отлично";
    numericGrade = 5;
}
else if (score >= 71 && score < 90)
{
    grade = "Хорошо";
    numericGrade = 4;
}
else if (score >= 51 && score < 70)
{
    grade = "Удовлетворительно";
    numericGrade = 3;
}
else if (score >= 0 && score < 50)
{
    grade = "Неудовлетворительно";
    numericGrade = 2;
}
else
{
    // Обработка некорректного ввода
    grade = "Ошибка";
    numericGrade = 0;
    Console.WriteLine("Ошибка: баллы должны быть от 0 до 100");
    return; // Выход из программы
}
// Вывод результата
Console.WriteLine("Результат");
Console.WriteLine($"Баллы: {score}");
Console.WriteLine($"Оценка: {grade} ({numericGrade})");
```

Запустите в терминале код:

dotnet run

В зависимости от ввода, вы получите соответствующий вывод:

```
└─>$ dotnet run
Определение оценки по баллам
Введите количество баллов (0-100): 56
Результат
Баллы: 56
Оценка: Удовлетворительно (3)
```

Практическое задание

1. Нарисуйте блок-схему проверки оценки
2. Протестируйте программу с разными значениями:
 - 95 (ожидается: Отлично)
 - 80 (ожидается: Хорошо)
 - 65 (ожидается: Удовлетворительно)
 - 40 (ожидается: Неудовлетворительно)
 - 110 (ожидается: Ошибка)
3. Сделайте три скриншота:
 - **step5_gradeDiagramLab17_FIO.png**
 - **step5_gradeCodeLab17_FIO.png**
 - **step5_gradeTestLab17_FIO.png** (результаты всех тестов)
4. Выполните команды:

```
cd ../../
```

```
git add .
```

```
git commit -m "Step 5: multiple conditions with if-else if-else"
```

```
git push
```

Шаг 6. Циклы: повторение действий

Цикл — это конструкция, позволяющая выполнять блок кода многократно.

Типы циклов в C#

1. **while** — цикл с предусловием (проверка ДО выполнения)
2. **do-while** — цикл с постусловием (проверка ПОСЛЕ выполнения)
3. **for** — цикл со счётчиком (известное число повторений)

Жизненный пример: Подъём по лестнице

Задача: Подняться на N-й этаж по лестнице

Алгоритм:

1. Начало

2. Узнать номер целевого этажа (N)
3. Установить текущий этаж = 1
4. **ПОКА** текущий этаж < целевого:
 - Подняться на 1 этаж выше
 - Увеличить счётчик текущего этажа
5. Конец

Создайте файл:

[touch diagrams/step6_stairs_while.drawio](#)

Нарисуйте блок-схему цикла while:



Создайте файл с кодом:

cd code/

dotnet new console --name Step6_StairsClimb

cd Step6_StairsClimb/

В файл **Step6_StairsClimb/Program.cs** добавьте следующий код:

```
// Начало
Console.WriteLine("Подъём по лестнице");
// Ввод данных
Console.Write("На какой этаж нужно подняться?: ");
int targetFloor = Convert.ToInt32(Console.ReadLine());
// Инициализация переменной цикла
int currentFloor = 1;
Console.WriteLine("Начинаем подъём!");
// Цикл while
while (currentFloor < targetFloor)
{
    .... currentFloor++; // currentFloor = currentFloor + 1
    .... Console.WriteLine($"Поднялись на {currentFloor}-й этаж");
}
// Вывод результата
Console.WriteLine($"Вы достигли {targetFloor}-го этажа!");
Console.WriteLine("Конец программы");
```

Запустите в терминале код:

dotnet run

В зависимости от ввода, вы получите соответствующий вывод:

```
└─>$ dotnet run
Подъём по лестнице
На какой этаж нужно подняться?: 3
Начинаем подъём!
Поднялись на 2-й этаж
Поднялись на 3-й этаж
Вы достигли 3-го этажа!
Конец программы
```

Цикл for (со счётчиком)

Когда заранее известно количество повторений, удобнее использовать цикл **for**.

Пример: Таблица умножения

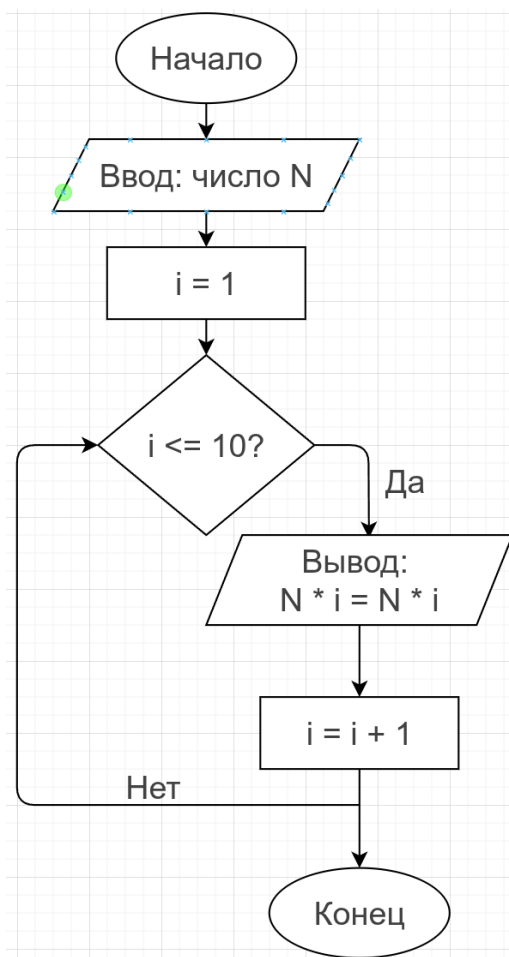
Задача: Вывести таблицу умножения для числа N.

Создайте файл:

cd ../../

touch diagrams/step6_multiplication_for.drawio

Нарисуйте блок-схему цикла **for**:



Создайте файл с кодом:

```
cd ../../
```

```
cd code/
```

```
dotnet new console --name Step6_MultiplicationTable
```

```
cd Step6_MultiplicationTable/
```

В файл **Step6_MultiplicationTable/Program.cs** добавьте следующий код:

```
// Начало
Console.WriteLine("Таблица умножения");
// Ввод данных
Console.Write("Введите число для таблицы умножения: ");
int number = Convert.ToInt32(Console.ReadLine());
Console.WriteLine($"Таблица умножения для {number}:");
// Цикл for
for (int i = 1; i <= 10; i++)
{
    int result = number * i;
    Console.WriteLine($"{number} x {i} = {result}");
}
// Конец
Console.WriteLine("Конец программы");
```

Запустите в терминале код:

dotnet run

В зависимости от ввода, вы получите соответствующий вывод:

```
└─>$ dotnet run
Таблица умножения
Введите число для таблицы умножения: 4
Таблица умножения для 4:
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40
Конец программы
```

Практическое задание

1. Нарисуйте две блок-схемы:
 - Цикл while (подъём по лестнице)
 - Цикл for (таблица умножения)
2. Запустите обе программы и протестируйте
3. Сделайте **четыре скриншота**:
 - **step6_whileDiagramLab17_FIO.png**
 - **step6_whileRunLab17_FIO.png**
 - **step6_forDiagramLab17_FIO.png**
 - **step6_forRunLab17_FIO.png**
4. Выполните команды:

```
cd ../../
```

```
git add .
```

```
git commit -m "Step 6: while and for loops with flowcharts"
```

```
git push
```

Шаг 7. Массивы: работа с коллекциями данных

Массив — это структура данных, позволяющая хранить несколько значений одного типа под одним именем.

Аналогия: Массив — это коробка с пронумерованными ячейками, где каждая ячейка хранит одно значение.

Основные понятия

- **Элемент массива** — отдельное значение в массиве
- **Индекс** — номер элемента (начинается с 0!)

- **Длина массива** — количество элементов

Пример: Массив оценок студента

Индекс 0	Индекс 1	Индекс 2	Индекс 3	Индекс 4
5	4	5	3	4

Практическое задание

1. Нарисуйте две блок-схемы:
 - Вывод элементов массива
 - Поиск максимального элемента
2. Создайте две программы на языке C#
3. Сделайте четыре скриншота:
 - **step7_arrayElementDiagramLab17_FIO.png**
 - **step7_arrayElementRunLab17_FIO.png**
 - **step7_arrayMaxDiagramLab17_FIO.png**
 - **step7_arrayMaxRunLab17_FIO.png**
4. Выполните команды:

git add .

git commit -m "Step 7 arrays"

git push

Шаг 8. Функции (методы)

Функция (метод) — это блок кода, выполняющий определённую задачу, который можно вызвать по имени.

Зачем нужны функции?

1. **Повторное использование кода** — написать один раз, использовать много раз
2. **Читаемость** — разбиение сложной программы на простые части
3. **Модульность** — каждая функция решает одну задачу
4. **Упрощение отладки** — легче найти ошибку в маленькой функции

Структура функции в C#

```

модификатор возвращаемый_тип ИмяФункции(параметры)
{
    // Тело функции
    return значение; // если функция возвращает результат
}
  
```

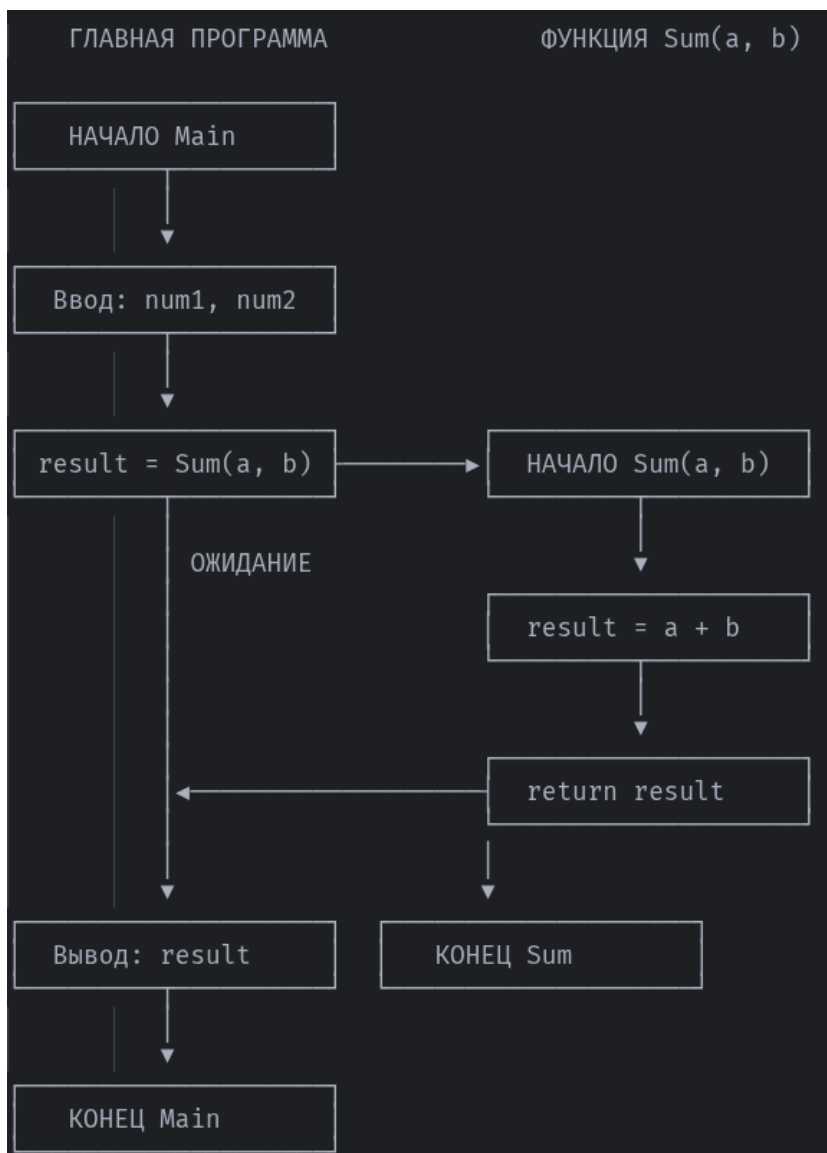
Пример:

```
static int Sum(int a, int b)
{
    int result = a + b;
    return result;
}
```

Практическое задание

1. Создать калькулятор с функциями для разных операций (сложение, вычитание, умножение, деление) на C#
2. Нарисуйте блок-схему функции с параметрами и возвращаемым значением.
3. Сделайте **три скриншота**:
 - [step8_functionDiagramLab17_FIO.png](#)
 - [step8_calculatorLab17_FIO.png](#)
 - [step8_primeLab17_FIO.png](#)

Пример реализации функции Sum(a, b):



4. Выполните команды:

git add .

git commit -m "Step 8: functions and modular programming"

git push

Финальный коммит

После завершения всех шагов выполните:

git add .

git commit -m "Completed Lab: Algorithms and Flowcharts"

git push

Самостоятельные задания

Теперь, когда вы освоили основы алгоритмов, попробуйте решить следующие задачи самостоятельно. Для каждой задачи:

1. Нарисуйте блок-схему в **Draw.io**
2. Напишите программу на C#
3. Протестируйте с разными входными данными
4. Сделайте скриншоты
5. Сделайте коммит в Git

Задание 1: Калькулятор ИМТ (Индекс Массы Тела)

Описание: Создайте программу для расчёта ИМТ и определения категории веса.

Формула: $\text{ИМТ} = \text{вес (кг)} / (\text{рост (м)})^2$

Категории:

- < 18.5 : Недостаточный вес
- 18.5-24.9: Нормальный вес
- 25-29.9: Избыточный вес
- ≥ 30 : Ожирение

Требования:

- Ввод веса и роста
- Расчёт ИМТ
- Определение категории
- Вывод рекомендаций

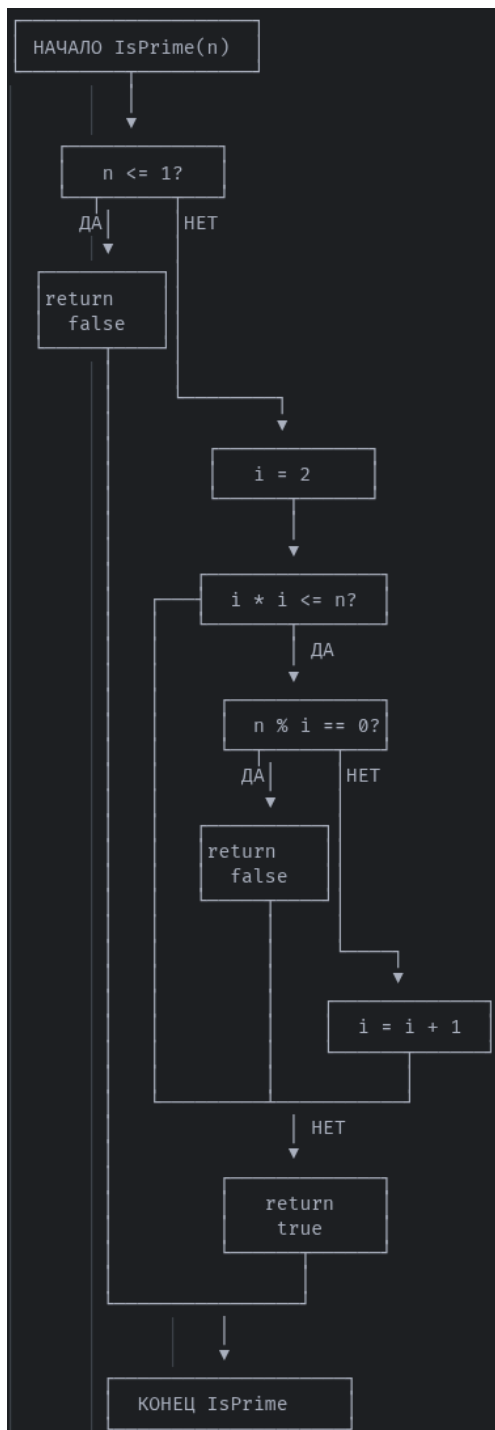
Файлы:

- [diagrams/task1_bmi.drawio](#)
- [examples/Task1_BMI.cs](#)
- [screenshots/task1_bmiLab17_FIO.png](#)

Задание 2: Создание функции проверки простого числа

Описание: Создайте и реализуйте функцию проверки простого числа `IsPrime(number)`.

Пример реализации:



Файлы:

- [diagrams/task2_isPrime.drawio](#)
- [examples/Task2_isPrime.cs](#)
- [screenshots/task2_isPrimeLab17_FIO.png](#)

Задание 3: Угадай число

Описание: Игра, где компьютер загадывает число от 1 до 100, а пользователь пытается угадать.

Правила:

- Компьютер загадывает случайное число
- Пользователь вводит попытку
- Программа говорит "больше" или "меньше"
- Счётчик попыток
- После угадывания — вывод статистики

Требования:

- Использовать цикл while
- Генерация случайного числа
- Обработка некорректного ввода
- Возможность играть снова

Файлы:

- [diagrams/task3_guess_game.drawio](#)
- [examples/Task3_GuessGame.cs](#)
- [screenshots/task3_guessGameLab17_FIO.png](#)

Задание 4: Оформление файла README.md

Задание: Добавьте описание в README.md.

Пропишите что вы изучили в ходе лабораторной работы, какое расширение использовали, что такое алгоритм, как составляются блок-схемы.